A language model of the Bitcoin trade tape is surprised by volatility

Navid Nabijou*

October 10, 2026

Abstract

We train a small causal transformer on a tokenised sequence of Bitcoin trades and ask whether its surprise tracks market volatility. Each trade event becomes one token that records its side, its size and the price move it causes. The model is trained on two months and scored on the following month, which holds approximately 20M events. Its rolling mean surprisal correlates with realised volatility at 0.56. A simple benchmark shows that part of this link is built in, because large price moves are rare tokens and also make up volatility. The model is nevertheless far less surprised than a benchmark that scores each event only by how rare its token is. It stays about 1.1 nats below the benchmark even in the most volatile tenth of the month.

1 Introduction

Financial markets alternate between calm and turbulent regimes. Large price changes tend to cluster in time (Cont, 2007), and the level of activity moves between low and high states. A model trained on the stream of trades could notice such a switch if it is surprised whenever the market departs from what it has learned. Perplexity, the exponential of the mean surprisal, is the standard summary of how surprised a language model is. We ask whether the perplexity of a transformer trained on the trade tape rises when the market is volatile.

We treat the tape as a language. Each trade event becomes one token. An event is one taker order, which is an order that executes immediately against orders already resting in the market. The token records the side of the order, its size and the price move it causes. No knowledge of finance is assumed, and Appendix A explains the terms used. The vocabulary has 182 tokens. A small causal transformer is trained on two months of Bitcoin trades and then frozen. We score every event of the following month by its surprisal. We average the surprisal over windows of 3000 events and compare the average with realised volatility over the same events.

The question is one of detection. We ask whether surprisal is high when volatility is high at the same time. We do not ask whether surprisal predicts later volatility.

The findings fall into two parts.

1. **The model is surprised by volatility.** Rolling mean surprisal and log realised volatility correlate at 0.56 over 20 659 283 test events, with a 95% block-bootstrap interval of 0.42 to 0.60. Surprisal is 0.15 nats higher in the high-volatility regime than elsewhere (Section 3).

2. **But less surprised than rarity alone would suggest.** Part of this link is built in, because large price moves are rare tokens and also make up volatility. A benchmark that scores each event only by the rarity of its token averages 4.33 nats per event, and the

*Queen Mary University of London. nn333@cantab.ac.uk

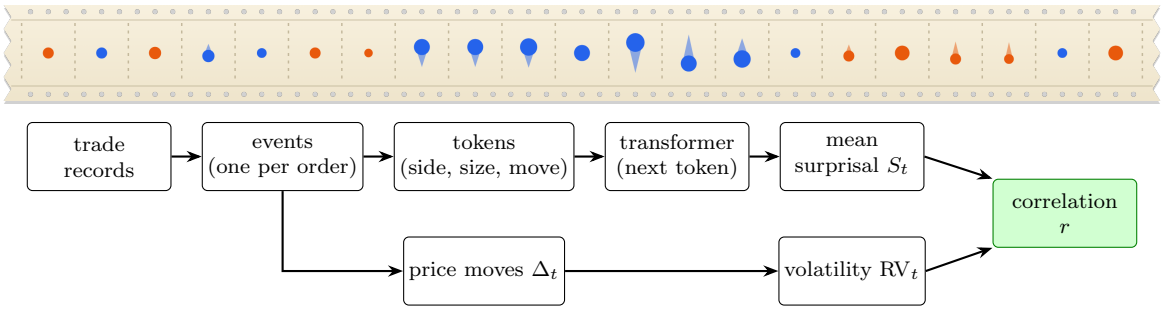


Figure 1: The design. Trade records are merged into events and each event becomes a token. A transformer trained on September and October scores each event of November by its surprisal. The rolling mean surprisal is compared with the realised volatility computed from the same events. The strip at the top shows a stretch of tokens: colour is the side, dot size is the trade size, and the tail is the price move.

model averages 3.22. The model stays about 1.1 nats below the benchmark even in the most volatile tenth of the data (Section 4).

Section 2 describes the design. Sections 3 and 4 give the two parts of the result. Section 5 offers labelled interpretation, Section 6 states the limitations, and Section 7 reviews related work. The appendices give the market terms, the tokeniser, the model, the statistics, the detailed results and the inference interface.

2 Method

2.1 Data and tokens

The data are the aggregate trades of BTCUSDT on Binance spot (Binance). BTCUSDT is Bitcoin traded against USDT, a token pegged to the US dollar, and spot means immediate settlement. Appendix A explains this and the other market terms we use. We use 91 days from 1 September to 30 November 2025 (UTC). September and October form the training period (61 days) and November is the test month (30 days). The split is chronological.

A taker order that is large enough to sweep several price levels appears as several consecutive records. We merge consecutive records into one event when they share the side and the timestamp, and the price continues in the aggressor's direction. This gives 51.8 million events from 101.8 million records (Appendix A).

Each event becomes one token with three fields.

- **Side:** buy or sell, according to which party took the liquidity.
- **Size:** the quantity traded, binned on a logarithmic scale with three bins per decade and 13 bins in total.
- **Price move:** the change in the last fill price from the previous event, in ticks of 0.01 USDT (the smallest price step). It falls into one of seven classes: flat, one tick up or down, and a mid or large move in each direction.

The vocabulary is $2 \times 13 \times 7 = 182$. The tokens carry no timestamps (Appendix B).

Figure 2 shows how we draw tokens. Each token is a comet: the colour is the side, the radius of the dot is the size bin, and the tail records the price move. The tail trails behind the dot, so a rising price puts the dot high with a tail below it, and a falling price does the opposite. The strip at the top of Figure 1 is a stretch of tokens drawn this way.



Figure 2: Key to the token icons, for a sample of five of the 13 size bins. Columns are the seven price-move classes, from a large fall to a large rise. Rows are size bins from small to large, and the dot radius grows with the bin. Blue marks a buy and orange a sell.

2.2 Model and scoring

The model is a decoder-only transformer (Vaswani et al., 2017; Radford et al., 2019) with six layers, width 192, six attention heads, a context of 512 events and 2 802 816 parameters. It is trained by next-token cross-entropy in a single pass over the training events. We train three seeds with one recipe and keep the seed with the lowest final training loss. The weights are then frozen, and no test data enter the choice (Appendix C).

The surprisal of event t is

$$s_t = -\log p_\theta(x_t \mid x_{t-h}, \ldots, x_{t-1}), \tag{1}$$

where the history length h lies between 448 and 511 events for every scored event (Appendix D). The rolling mean over the last $N = 3000$ events is $S_t = \frac{1}{N}\sum_{i=t-N+1}^{t} s_i$.

2.3 Volatility and the statistic

Realised volatility in event time is the root mean square of the price moves Δ_i over the last $M = 3000$ events,

$$\mathrm{RV}_t = \left(\frac{1}{M}\sum_{i=t-M+1}^{t} \Delta_i^2\right)^{1/2}. \tag{2}$$

Because $N = M$, the two series summarise the same events. The high-volatility regime is RV_t above the 90th percentile of the training period, which is 251 ticks. The regime flag is used for shading and for the comparison in Section 3. It does not enter the headline statistic.

The headline statistic is the Pearson correlation r between S_t and $\log \mathrm{RV}_t$ over the test month. Both series are averages over overlapping windows, so neighbouring values are almost identical. We therefore compute a 95% interval by a moving-block bootstrap with blocks of 1 000 000 events and 2 000 replicates (Appendix F). All settings above were fixed before the model was scored.

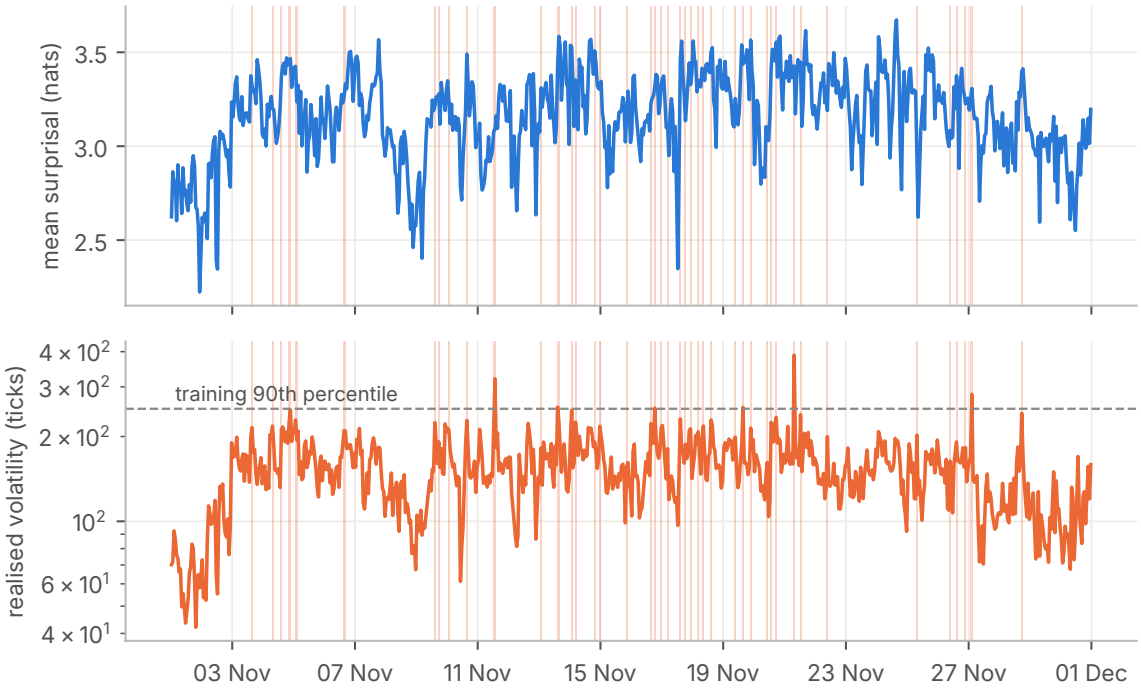


Figure 3: Hourly averages over the test month, November 2025. Top: mean surprisal. Bottom: realised volatility on a log scale, with the training 90th percentile dashed. Shading marks hours in which at least a fifth of the events exceed that threshold.

3 The model is surprised by volatility

Figure 3 shows the two series over the test month. Surprisal and volatility rise and fall together. The correlation is 0.56 with a 95% interval of 0.42 to 0.60.

The regime comparison gives the same picture. Mean surprisal is 3.35 nats where volatility is above the training threshold and 3.21 nats elsewhere, a difference of 0.15 nats. Figure 4a groups the events by volatility decile. Surprisal rises from 2.8 nats in the calmest decile to 3.40 in the ninth. It then does not rise further, and it is 3.36 in the tenth. The daily means of the 30 test days correlate at 0.90 (Figure 4b). That figure is a description of 30 points and not a test.

We make no claim that surprisal leads volatility. Shifting one series against the other lowers the correlation on both sides of zero (Appendix F). That is expected from overlapping windows and carries no information about lead or lag.

4 But less surprised than rarity alone would suggest

To judge the model we need a reference. Our reference is the rarity benchmark. It scores each test event by the surprisal of its token under the token frequencies of the training period, with no context at all.

The model is much less surprised than the benchmark. Averaged over the test month, the benchmark scores 4.33 nats per event and the model scores 3.22 (Figure 5a). The gap is 1.2 nats in the calmest decile and 1.1 nats in the most volatile one. What the model learned from September and October therefore still applies when the market is volatile.

Part of the link in Section 3 is built in. Large price moves are rare tokens, and they are also what volatility is made of. The rolling mean of the benchmark correlates with volatility at 0.68, above the model's 0.56. The correlation of Section 3 therefore does not by itself show

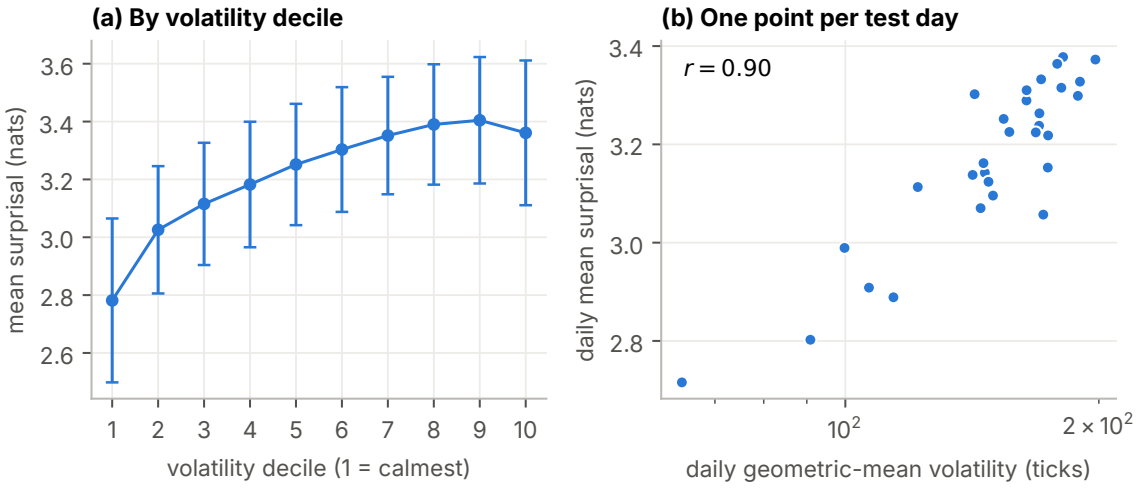


Figure 4: (a) Mean surprisal by decile of realised volatility over the test month. Bars show one standard deviation of the rolling mean within the decile. (b) Daily means of surprisal against daily geometric-mean volatility.

that the model has learned anything about sequences. The decomposition in Figure 5b shows where the link comes from. It scores each field of the token separately.

- **Price move:** the correlation is 0.49. This is expected by construction, because volatility is computed from price moves.
- **Size:** the correlation is 0.58. Size does not enter the volatility measure, so this part is market behaviour: trades are larger when the market is volatile.
- **Side:** there is no correlation (-0.01).

The contributions of size and price move overlap, because large trades often cause large moves, and they should not be added.

Little of the model's gain comes from the volatility level. A model that knew only the volatility decile before each event would gain 0.04 nats over the benchmark. The transformer gains 1.11 nats. Knowing how volatile the market is therefore explains about 4% of the improvement (Appendix G).

5 Discussion

This section is interpretation. We state it because it bears on how the results should be read.

Why surprisal tracks volatility. A natural reading is that volatile markets are intrinsically less predictable. In the test month the entropy of the token distribution, given the volatility decile before the event, rises from 3.97 to 4.41 nats across the deciles (Appendix G). Even a model that knew the volatility level perfectly would be more surprised in volatile periods. We have not tested this reading beyond that calculation.

Why the top decile does not rise. The largest price-move class is open-ended above 129 ticks, about 1.29 USDT. A very large move therefore falls in the same token as a moderately large one, and surprisal cannot grow once moves saturate. We have not tested whether this explains the flattening.

What the model has learned. Most of the model's gain over the benchmark is local. The transformer gains 1.11 nats over the benchmark. A count-based model that sees only the previous token reaches 73% of that gain (Appendix G). This model is a table of next-token

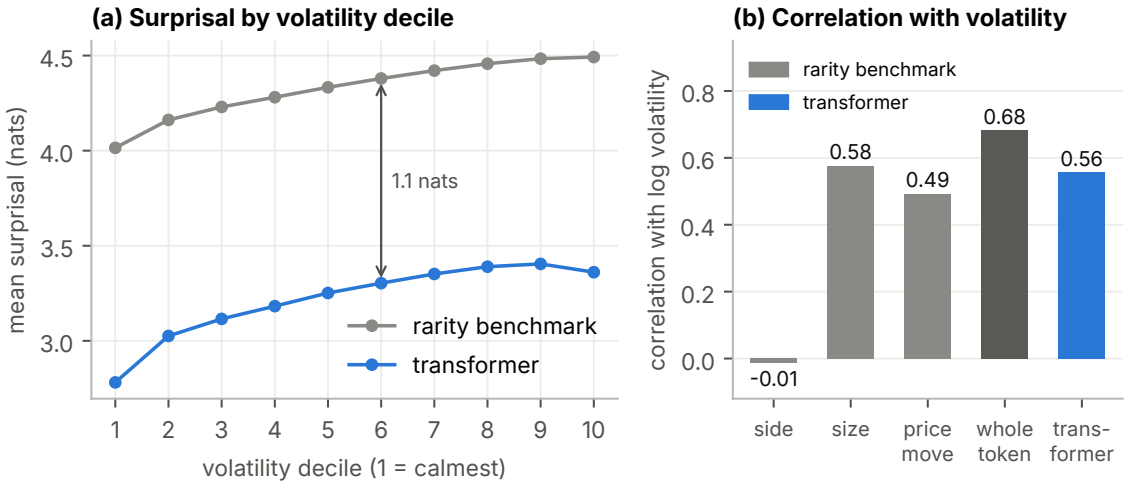


Figure 5: (a) Mean surprisal by volatility decile for the rarity benchmark and the transformer. (b) Correlation with log volatility of the rolling mean rarity of each token field, and of the whole token.

frequencies given the previous token, estimated on the whole training period. It is not a neural network. The training loss of the transformer falls by about 0.14 nats as the history grows from a few events to several hundred (Appendix C). We do not claim that the model uses long context.

6 Limitations

One model, one month, one market. Only the selected seed was scored. The final training losses of the three seeds differ by 0.0129 nats, and the variation of the correlation across training runs is not measured. The interval reflects resampling of the test month only. The data are one asset on one venue over one month. The test month is milder than the training period: 7.6% of test events are in the high-volatility regime, against 10% of training events, which is the definition of the threshold.

Interval and windows. The interval depends on the block length. The bootstrap standard error rises with block length and levels off near the length we used (Appendix F). We did not vary N or M.

Tokenisation. We used one tokenisation and did not vary the bins. The benchmark's rarity depends on the training mix of tokens.

Scope. The result is simultaneous. We do not test lead or lag, and we do not compare with standard volatility forecasting models such as GARCH, which predict volatility from past price changes.

7 Related work

Several works build generative models of order flow or of time series from tokens.

- Nagy et al. (2023) tokenise limit order book messages by digit groups and train a state-space model on them.
- Linna et al. (2025) train a BERT-style model with one token per message.
- Ansari et al. (2024) tokenise general time series by scaling and quantisation.

These works study generation or forecasting. We did not find prior work that uses the surprisal of such a model as an indicator of the volatility regime. Our search was not exhaustive.

The volatility measure is realised volatility (Andersen et al., 2003), here computed in event time. Event time is a transaction clock, and the cumulative number of trades is a better stochastic clock than volume for obtaining near-normal returns (Ané and Geman, 2000). Volatility clustering and the other stylised facts are reviewed by Cont (2001, 2007). Trading volume is positively related to the size of price changes (Karpoff, 1987), which is consistent with the link between trade size and volatility in Section 4. The moving-block bootstrap is due to Künsch (1989). Automatic block-length rules exist (Politis and White, 2004; Patton et al., 2009), and we chose the length from the dependence of the series (Appendix F).

8 Conclusion

A small transformer trained on two months of Bitcoin trades is more surprised when the market is more volatile. The correlation is 0.56 on the test month, with a 95% interval of 0.42 to 0.60. Part of that link is built in, because large moves are rare events and also make up volatility. The model is nevertheless about 1.1 nats less surprised than a benchmark that looks only at token rarity, and the margin holds in the most volatile decile. Three follow-up studies would extend the result.

- A lead test would ask whether surprisal rises before volatility does.
- Scoring the other seeds would measure the variation of the result across training runs.
- Other assets and months would test whether the result is specific to this market.

Acknowledgements

Beyond the initial project design, this project's code, analysis and this write-up were developed in collaboration with Claude (Anthropic).

References

Torben G. Andersen, Tim Bollerslev, Francis X. Diebold, and Paul Labys. Modeling and forecasting realized volatility. *Econometrica*, 71(2):579–625, 2003.

Thierry Ané and Hélyette Geman. Order flow, transaction clock, and normality of asset returns. *The Journal of Finance*, 55(5):2259–2284, 2000.

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.

Binance. Binance public data. `https://github.com/binance/binance-public-data`. Data files at `https://data.binance.vision`. Accessed October 2026.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Rama Cont. Volatility clustering in financial markets: Empirical facts and agent-based models. In Gilles Teyssière and Alan P. Kirman, editors, *Long Memory in Economics*, pages 289–309. Springer, 2007.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

Jonathan M. Karpoff. The relation between price changes and trading volume: A survey. *Journal of Financial and Quantitative Analysis*, 22(1):109–126, 1987.

Hans R. Künsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, 1989.

Eljas Linna, Kestutis Baltakys, Alexandros Iosifidis, and Juho Kanniainen. LOBERT: Generative AI foundation model for limit order book messages. *arXiv preprint arXiv:2511.12563*, 2025.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Peer Nagy, Kang Li, Sascha Frey, Anisoara Calinescu, Jakob Foerster, Silvia Sapora, and Stefan Zohren. Generative AI for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. In *Proceedings of the ACM International Conference on AI in Finance (ICAIF)*, pages 91–99, 2023.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Andrew Patton, Dimitris N. Politis, and Halbert White. Correction to "Automatic block-length selection for the dependent bootstrap" by D. Politis and H. White. *Econometric Reviews*, 28 (4):372–375, 2009.

Dimitris N. Politis and Halbert White. Automatic block-length selection for the dependent bootstrap. *Econometric Reviews*, 23(1):53–70, 2004.

Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the Transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

A Market terms and data

Market. A crypto exchange such as Binance runs a marketplace where people trade one asset for another. BTCUSDT is the market in which Bitcoin (BTC) is bought and sold for USDT, a token whose value is pegged to the US dollar. The price is the number of USDT per BTC. A spot market settles trades immediately, as opposed to derivatives such as futures.

Order book, maker and taker. The exchange keeps an order book, a list of orders that are waiting to trade. Each waiting order states a side (buy or sell), a price and a quantity, and is also called a limit order. Buy orders are bids and sell orders are asks. A trade happens when a new order meets a waiting order at an acceptable price.

- A *maker* order waits in the book and provides liquidity, meaning the chance for others to trade.
- A *taker* order executes at once against waiting orders and removes liquidity. A market order is a taker order.
- The taker is the *aggressor* of a trade. The side of a trade is the side of its taker. A buy taker consumes the lowest asks, which tends to raise the price, and a sell taker consumes the highest bids, which tends to lower it.

Sweeping. A waiting order has only a limited quantity. A taker order larger than the quantity at the best price continues to the next price, and so on. It is said to sweep several price levels. The exchange reports the fill at each level as a separate trade, so one taker order can produce several records at the same instant and at steadily worse prices.

Aggregate trade records. Binance publishes aggregate trades, in which fills of one taker order at the same price are already combined. Each record has the following fields.

- An aggregate trade identifier that increases with time.
- A price and a quantity.
- A timestamp.
- A flag for whether the buyer was the maker. The aggressor is the seller when the buyer was the maker, and the buyer otherwise.

Units. A tick is the smallest price step, which is 0.01 USDT for BTCUSDT. We write the price move of an event in ticks. We count quantities in lots of 10^{-5} BTC. Prices and quantities are held as integers in these units, so that no comparison involves rounding.

Volatility. Volatility measures how large the price changes are. Realised volatility is computed from observed changes and not from a model. We define it in event time in Section 2. Markets move between calm periods and volatile ones, which are called regimes, and large changes tend to cluster together. We call a period a high-volatility regime when volatility is above a fixed threshold.

Source. The files are the daily `aggTrades` archives for BTCUSDT spot from `data.binance.vision` (Binance). Each file has a checksum, and every file was verified. From 1 January 2025 the spot timestamps are in microseconds.

Cleaning.

- Records with zero price or zero quantity are audit duplicates and are dropped.
- Records are ordered by aggregate trade identifier.

Merging records into events. An event is one taker order. A record joins the current event when it has the same side as the previous record, the same timestamp, and a price that continues strictly in the aggressor's direction. Continuing means higher for a buy and lower for a sell. Each day is processed on its own, and an order split by midnight within one microsecond would be cut in two, which is negligible.

Table 1 gives a worked example. Four records give two events. The first three records are one buy order that sweeps three prices. The fourth record is a sell.

Record	Price	Quantity (BTC)	Timestamp	Side	Event
1	82 000.10	0.0020	t	buy	1
2	82 000.11	0.0150	t	buy	1
3	82 000.13	0.0400	t	buy	1
4	82 000.12	0.0010	$t+1$	sell	2

Table 1: Merging example. Event 1 has size 0.0570 BTC and last price 82 000.13. Its price move is measured against the last price of the previous event.

Counts. The 91 files hold 101,818,559 records, which merge into 51,819,298 events (1.96 records per event). The first event has no previous price and is dropped. This leaves 31 157 015 training events and 20 662 282 test events.

B Tokeniser

Fields. A token is $(\text{side}, \text{size}, \text{move})$, coded as

$$\text{id} = (\text{side} \cdot n_{\text{size}} + \text{size}) \cdot 7 + \text{move}, \tag{3}$$

with side 0 for sell and 1 for buy, size from 0 to $n_{\text{size}} - 1$ and move from 0 to 6. With $n_{\text{size}} = 13$ the vocabulary is 182.

Price move. The move is the change in the last fill price of the event from the last fill price of the previous event, in ticks. Table 2 (left) lists the seven classes. A move of 2 ticks or more is mid or large. It is large when it exceeds 129 ticks, which is the median of such moves in the training period, and mid otherwise. The boundaries are fitted on the training period only.

Size. Sizes are binned on a logarithmic scale with edges at $5 \times 10^{k/3}$ lots for $k = 1, \ldots, 15$, which is three bins per decade. The tail rule merges an outer bin that holds less than 1% of the training events into its inner neighbour, and repeats until the outermost bin holds at least 1%. This reduces the 16 raw bins to 13. Table 2 (right) lists the final bins.

Icons. Figure 2 and the strip in Figure 1 draw each token as a dot with a tail. Colour is the side and dot radius is the size bin. The tail points against the move, and its length is 4, 8 or 14 units on a 32-unit tile for a move of one tick, a mid move or a large move. The tail extends that distance beyond the edge of the dot, whatever the size of the dot.

Price move	Share of training events
down, large	4.3%
down, mid	4.4%
down, 1 tick	14.3%
flat	54.3%
up, 1 tick	14.4%
up, mid	4.1%
up, large	4.2%

Bin	Size range (BTC)	Share of training events
1	below 0.00011	13.1%
2	0.00011–0.00023	9.4%
3	0.00023–0.0005	9.5%
4	0.0005–0.0011	12.5%
5	0.0011–0.0023	10.2%
6	0.0023–0.005	9.9%
7	0.005–0.011	9.9%
8	0.011–0.023	8.8%
9	0.023–0.05	6.2%
10	0.05–0.11	5.1%
11	0.11–0.23	2.7%
12	0.23–0.5	1.6%
13	above 0.5	1.2%

Table 2: Left: the seven price-move classes and their shares of training events. Right: the size bins and their shares.

C Model and training

Architecture. The model is a decoder-only transformer with pre-normalisation (Xiong et al., 2020), GELU activations (Hendrycks and Gimpel, 2016) and learned absolute position embeddings. The input and output embeddings are tied (Press and Wolf, 2017). There is no dropout. Table 3 lists the settings.

Setting	Value
Layers, width, heads	6, 192, 6
Context	512 events
Vocabulary	182
Parameters	2 802 816
Optimiser	AdamW (Loshchilov and Hutter, 2019), $\beta = (0.9, 0.95)$
Peak learning rate	10^{-3}, 100 warm-up steps, cosine decay to 10%
Weight decay	0.1, on weight matrices and embeddings only
Gradient clipping	1.0
Batch	32 chunks of 512 events (16 384 tokens)
Training	one pass, 1901 steps, chunks shuffled
Seeds	0, 1, 2
Software	PyTorch (Paszke et al., 2019)

Table 3: Model and training settings.

Training data. The training stream is cut into non-overlapping chunks of 512 events, giving 60,853 chunks, and the chunks are shuffled with a fixed seed. The input is 512 consecutive events and the target is the same events shifted by one. The last partial batch of 21 chunks is dropped, so the final 10,752 events of the training period are not used as targets. The single pass means that every batch is new data, and there is no separate validation set.

Seed choice. Table 4 gives the final training loss of each seed, the mean over the last tenth of the steps. Seed 2 has the lowest loss and is kept. The choice uses training loss only. Figure 6a shows the three loss curves, which are almost indistinguishable.

Seed	Final training loss (nats)	Training time (min)	Selected
0	3.1227	68	
1	3.1295	73	
2	3.1166	73	yes

Table 4: Final training loss by seed. The spread is 0.0129 nats. The runs were on a laptop CPU with four cores and no GPU.

Loss by position. Figure 6b shows the mean training loss at each position within a chunk, averaged over the whole pass, so it includes the early steps. The loss falls from about 3.34 nats after a few events of history to about 3.20 nats after a few hundred. Scored events have between 448 and 511 events of history and so lie on the flat part.

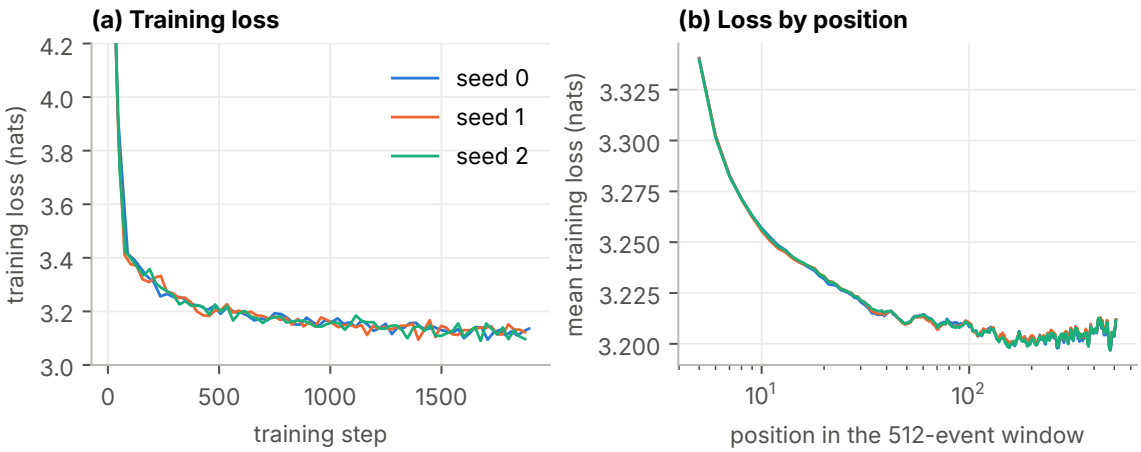


Figure 6: (a) Training loss of the three seeds, each point is the mean of the last 20 steps, sampled about once a minute. (b) Mean training loss by position in the 512-event chunk, smoothed over 8 positions, with a log axis.

D Scoring

The model reads at most 512 events. If the test stream were cut into plain chunks of 512, the first event of a chunk would have no history and the last would have 511. Surprisal falls as history grows, so the series would show a spurious sawtooth.

Instead, a window of 512 events slides over the stream with a stride of 64 events, and only the last 64 events of each window are scored. Let the extended stream be the last 511 training events followed by the test events. The training events are context only and are never scored. Test event j (counting from 0) is scored in window $k = \lfloor j/64 \rfloor$. It is predicted from $448 + (j \bmod 64)$ events of history, which lies between 448 and 511. The test month has 322,849 windows. A stride of 1 would give every event 511 events of history, but it would take about 5.5 days on our laptop, against about two hours for a stride of 64.

The implementation is checked against a brute-force computation of $-\log p$ for individual events, and a scoring run that is stopped and resumed gives the same result as an uninterrupted run.

E Volatility and the regime flag

Realised volatility is computed on the continuous stream of training and test events. A window near the start of the test month reaches back into the end of the training period, and uses only events that precede the one measured. The regime flag is RV_t above the 90th percentile of RV_t over the training events with a full window, which is 251 ticks. It flags 10.0% of training events and 7.6% of test events. The window $M = 3000$ was chosen on 23 training days, before any test data were scored. It balances noise from single large sweeps against resolution in time. In the headline statistic RV_t is floored at 10^{-3} ticks before the logarithm.

F Statistics

Moving-block bootstrap. Let $(x_t, y_t)_{t=1}^{n}$ be the series of S_t and $\log \mathrm{RV}_t$ over the test month, with $n = 20\,659\,283$. With block length b, the blocks are all windows of b consecutive pairs. A replicate has three steps.

1. Draw $\lceil n/b \rceil$ block starts uniformly with replacement.
2. Glue the blocks and cut the result to length n.
3. Recompute the correlation.

We use $b = 1\,000\,000$ and 2 000 replicates. The interval is the 2.5th and 97.5th percentiles of the replicates (Künsch, 1989). Each replicate needs only the five sums $\sum x$, $\sum y$, $\sum x^2$, $\sum y^2$ and $\sum xy$ over its blocks. These are differences of prefix sums, evaluated at the few thousand positions that are needed, so 2 000 replicates over 20 659 283 events run in seconds.

Block length. The length $b = 1\,000\,000$ is about one and a half days of the test month, which gives about 21 blocks. It was fixed before scoring from the autocorrelation of $\log \mathrm{RV}$ in the training period. Figure 7a and Table 5 (left) show the bootstrap standard error as a function of block length on the test series. The error rises with block length and levels off near the length used, at 0.045. Automatic rules for the block length exist (Politis and White, 2004; Patton et al., 2009), and we did not use them. Table 5 (right) gives the autocorrelation of the two series.

Block length	Std. error	95% interval
1 000	0.007	[0.54 0.57]
10 000	0.017	[0.52 0.59]
100 000	0.030	[0.49 0.61]
300 000	0.040	[0.46 0.62]
1 000 000	0.045	[0.42 0.60]
2 000 000	0.041	[0.41 0.57]

Lag	$\log \mathrm{RV}$	Surprisal
3 000	0.67	0.74
30 000	0.46	0.48
300 000	0.16	0.15
1 000 000	0.05	0.06

Table 5: Left: bootstrap standard error and 95% interval of the correlation by block length. Right: autocorrelation of $\log \mathrm{RV}$ and of the mean surprisal.

Shifted series. Figure 7b recomputes the correlation after shifting the surprisal series against the volatility series. It is highest at zero shift and falls on both sides. Windows overlap, so a shift of less than N events keeps most of the window in common. The figure is not a lead test.

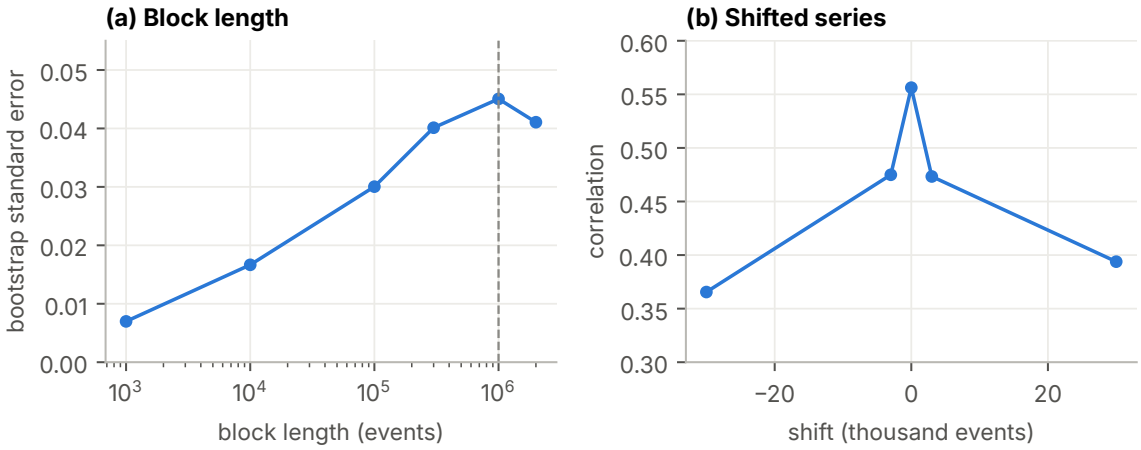


Figure 7: (a) Bootstrap standard error of the correlation by block length. The dashed line marks the length used. (b) Correlation after shifting the surprisal series by the given number of events.

G Rarity benchmark and further results

Definition. Let c_v be the count of token v in the training stream, with 0.5 added to every count. The rarity of a token is $-\log\big(c_v/\sum_u c_u\big)$. The rarity benchmark scores each test event by the rarity of its token. Its rolling mean over N events is compared with volatility exactly as the model's is. The benchmark uses the training period only.

Ladder of scoring rules. Table 6 adds a model that sees only the previous token. It is a table of next-token frequencies given the previous token, counted over the whole training period, and it is not a neural network. The counts are smoothed towards the token frequencies, by adding $\alpha \times 182$ pseudo-counts spread according to those frequencies. We chose $\alpha = 0.5$ by fitting the counts on the first 85% of the training period and scoring the last 15%, and then refitted the counts on all of it. On the held-out part, any α between 0.1 and 2 gives a mean surprisal within 0.001 nats of the best. The slope is the change of the rolling mean surprisal in nats per unit of log RV. Better models are less surprised on average. Their correlation with volatility is lower. Their slopes are not lower, so the lower correlation comes from more spread in the surprisal that is unrelated to volatility.

Scoring rule	Mean surprisal (nats)	Correlation with log RV	Slope (nats per unit)	Flagged minus rest (nats)
Token rarity (no context)	4.33	0.68	0.35	0.18
Previous token only	3.51	0.63	0.45	0.21
Transformer (512 events)	3.22	0.56	0.42	0.15

Table 6: Three scoring rules on the test month. "Flagged minus rest" is the difference in mean rolling surprisal between flagged and unflagged events.

By volatility decile. Table 7 gives the numbers behind Figure 5a. Figure 8 adds a model that knows only the volatility decile before each event. Its score is the empirical entropy, in nats, of the token distribution among events whose preceding volatility lies in the decile, computed on the test month. The entropy rises from 3.97 nats in the calmest decile to 4.41 in the most volatile.

Decile	RV range (ticks)	Transformer	Rarity	Gap
1	15–103	2.78	4.02	1.23
2	103–123	3.03	4.16	1.14
3	123–138	3.12	4.23	1.11
4	138–150	3.18	4.28	1.10
5	150–162	3.25	4.33	1.08
6	162–174	3.30	4.38	1.08
7	174–187	3.35	4.42	1.07
8	187–204	3.39	4.46	1.07
9	204–236	3.40	4.48	1.08
10	236–3258	3.36	4.49	1.13

Table 7: Mean surprisal in nats by volatility decile (deciles of the rolling window volatility RV). "Gap" is rarity minus transformer.

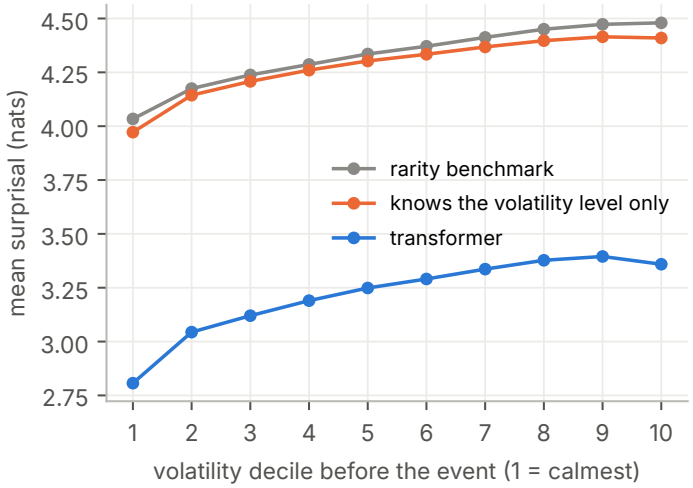


Figure 8: Mean surprisal by the volatility decile before the event. A model that knew only the volatility decile gains little over the rarity benchmark.

Token fields. Table 8 gives the correlations behind Figure 5b. In the calmest decile 10% of events have a mid or large move. In the most volatile decile the share is 26%.

Rarity of the token's …	Correlation with log RV
side	−0.01
price move	0.49
trade size	0.58
whole token	0.68

Table 8: Correlation with log RV of the rolling mean rarity of each field of the token.

By token class. Figure 9 gives the mean surprisal for each price-move class and each size bin. The model is below the benchmark in every class. Flat moves are the least surprising, and large trades are the most surprising.



Figure 9: Mean surprisal on the test month by (a) price-move class and (b) size bin, for the rarity benchmark and the transformer.

H Inference interface

The repository includes a small interface that continues a tape with the trained model. It is a demonstration and lies outside the claim tested in the paper. It computes no surprisal and changes no result. It is started with `python gui.py` and runs as a local web page. It reads only the weights, the tokeniser and a file of prompt excerpts, all kept in the repository, so it needs neither the downloaded data nor a retrained model.

Starting sequence. The user chooses one of three sources.

- **A stored excerpt.** The file holds ten real stretches of the test month, each with 511 events and the 256 events that followed. Five were drawn at random from stretches in the high-volatility regime and five from other stretches. They were drawn by a fixed rule and not picked by eye.
- **A time in the test month.** If the tokenised test month is on disk, any time in it can be chosen.
- **A sequence built by hand.** The user clicks side, size and price move to add one icon at a time, for instance to try a run of large sell orders.

Prompts come from the test month only, so the model has not seen them.

Generation. The model reads the sequence and computes a distribution over the next token. The interface samples one token from it, appends it, and repeats. There are four controls.

- The number of events to generate.
- The sampling temperature. A temperature of 1 samples from the model's own distribution, lower values stay near the likeliest tokens, and higher values wander.
- The number of continuations, each drawn as its own row.
- A random seed.

Every token is drawn with the icon of Figure 2. Generated events are drawn on a tinted background to the right of a divider. When the prompt is a stored excerpt, the events that

really followed are drawn underneath. Hovering over an icon gives its side, size range and price-move class.

Example. Figure 10 is a screenshot of the interface. The prompt is a stretch of the test month that starts at 17:20 UTC on 9 November 2025, when realised volatility at the end of the prompt was 392 ticks, above the regime threshold of 251 ticks. The model generated two continuations of 18 events at temperature 1. The prompt ends with large buys that carry long tails. The first continuation begins with two such tokens and then turns to small sells. The second begins with small tokens and has larger buys later. The two samples differ from each other and from the events that really followed. This is expected, because each token is a random draw from the model's distribution. The example is an illustration of the interface. We did not select it for its quality, and it is not evidence for any claim in the paper.

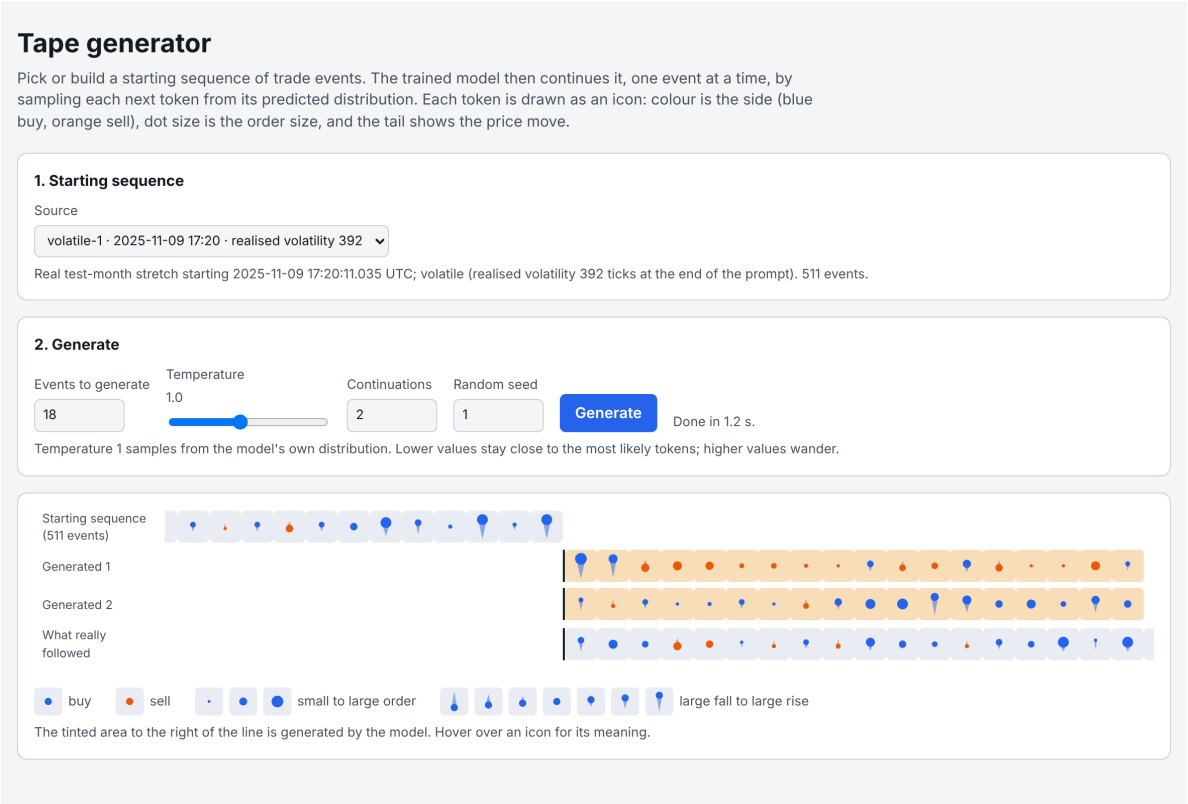


Figure 10: Screenshot of the inference interface. Top: the chosen excerpt and the generation controls. Bottom: the end of the 511-event starting sequence, two sampled continuations of 18 events on a tinted background, and the first events that really followed.

I Reproducibility

The whole study runs from one script, `python run.py`. It performs these steps in order.

1. Download and verify the data.
2. Build events and tokens.
3. Train the three seeds and select one.
4. Score the test month, evaluate, and export the weights.

It writes a checkpoint every 10 minutes of wall time, so that an interrupted run can resume, and it prints a status line with an estimated time to completion every minute. On a laptop with a four-core Intel i5-1038NG7 CPU and no GPU, a full run takes about six hours of computation. Each seed trains for about 71 minutes and the scoring takes about two hours.

The weights of the selected seed, the fitted tokeniser and a small file of prompt excerpts are in the repository, with a local interface for generating continuations of a tape. The figures, numbers and tables of this paper are produced from the run outputs by `paper/make_figures.py`. The screenshot of Appendix H is made by `paper/make_gui_screenshot.py`. The text quotes numbers only through the generated files, so the paper cannot disagree with the run. The rarity benchmark and the other descriptive checks of Appendix G are computed in that script.